1. Analyze two GenAI programs:

Meta Muse Spark 1.1 | Thinking

Claude Sonnet 5 | Medium + Thinking

2. How you accessed them:

I used both of them on their own respective websites

3. Prompt used

Objective

Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

Requirements

• Use a fixed secret number (for example, 7).
• Use input() to read the user's guess and convert it to an integer.
• Use a while or for loop to track the 3 attempts.
• Use if-elif-else statements to check if the guess is too high, too low, or correct.
• Use break to exit the loop early if the user guesses correctly.
• Print a final win or lose message using print().

Example Output

Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 5
Too low! Try again.
Attempt 2/3. Enter your guess: 8
Too high! Try again.
Attempt 3/3. Enter your guess: 7
Correct! You win!
Ex

Meta Muse Spark 1.1:

```c
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d/3. Enter your guess: ", attempt);

        // read integer from user
        if (scanf("%d", &guess) != 1) {
            printf("Invalid input. Please enter a number.\n");
            return 1;
        }

        if (guess == secret) {
            printf("Correct! You win!\n");
            break;
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }

        // if this was the last attempt and they didn't win
        if (attempt == 3) {
            printf("You lose! The number was %d.\n", secret);
        }
    }

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int secret = 7;      // Fixed secret number
    int guess;
    int attempts = 3;
    int won = 0;         // Flag to track if the user guessed correctly

    printf("Guess a number between 1 and 10.\n");

    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d/%d. Enter your guess: ", i, attempts);
        scanf("%d", &guess);

        if (guess > secret) {
            printf("Too high! Try again.\n");
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Correct! You win!\n");
            won = 1;
            break;   // Exit the loop early on a correct guess
        }
    }

    if (!won) {
        printf("You lose! The number was %d.\n", secret);
    }

    return 0;
}
```

2. The two GenAI examples include one other difference
   - What is the difference?

Claude uses a if(!won) statement while Meta Spark used if (attempt == 3) {, other than that logic is pretty very close to each other meaning that these agents ended up doing a pretty good job.

Analyze the difference in terms of:

a. Execution time

Meta's code may be marginally faster because it does not need to assign a value to the won variable or perform the additional if (!won) check after the loop. However, the difference is extremely small and would not be noticeable in practice. Most of the execution time in this program comes from scanf() and printf(), rather than the simple conditional logic.

b. Space complexity

Both programs have O(1) space complexity because they use a fixed number of variables. Meta's code does use slightly less memory because it does not need the additional won variable, but this difference is extremely small.
Claude's code

c. Maintainability

There are fewer variables to keep track of and the core of the logic is simpler than Claude's, however if the program expanded into something greater than this simple logic clause's code might be better.

3. Which of the two GenAI code examples would you select? Why?
   - I ended up selecting meta's code as the one that I used to make the code better by adding proper error handling. In my eyes the logic was more clear because Claude's code was more verbose than it needed to and achieve the scope of the project better.

4. How would you improve the following for the one you selected

a. Correctness

Meta's code ended up being more correct by adding error handling despite the prompt not saying to include it, this means that this agent may be more aware of aspects that the engineer may not think about meaning it understands the scope of the project better

b. Execution time

There is very little that needs to be optimized in terms of execution time. The main performance cost comes from user input and output through scanf() and printf(), not the conditional statements. Replacing scanf() with a faster input method could improve performance, but for such a small interactive program the difference would be negligible.

c. Space complexity

The code already has O(1) space complexity because it only uses a fixed number of variables. There is not much to optimize in terms of space. Removing variables could technically reduce memory usage slightly, but the amount of memory saved would be insignificant and could make the code less readable or functional.

d. Maintainability

The logic is more simple and more readable with Meta's version, however the attempts should be used more rather than hard coding it. Meaning that Claude's code does have some validity to the actions it selected and its choices.

Overall the agents did a great job in programming this intro in C course